

SOP (Standard Operating Procedure) Template based on the large DB Migration which can be dropped into internal runbooks, change records, or client documentation with minimal modification.

SOP: Large Database Migration Using Distributed Availability Groups

Target: Minimal Downtime, Planned Zero Data Loss

Scope: SQL Server 2019 → SQL Server 2022

Scale: Multi-TB (validated at 6TB)

Method: Distributed Availability Groups (DAGs)

UI: T-SQL only (no GUI dependencies)

1. Purpose

This SOP defines a repeatable, low-risk process for migrating large, business-critical SQL Server databases between clusters using **Distributed Availability Groups**, achieving:

- Planned zero data loss (RPO = 0 at cutover)
- Minimal application downtime
- Deterministic cutover with explicit data-loss gates
- No reliance on SSMS GUI workflows

2. Preconditions and Assumptions

2.1 Infrastructure

- Source environment:
 - SQL Server 2019
 - Existing Always On Availability Group (AG #1)
 - Two or more replicas
- Target environment:
 - New Windows cluster
 - SQL Server 2022
 - New Always On Availability Group (AG #2)
 - Identical database names
- Reliable network connectivity between clusters

2.2 Operational Requirements

- Vendor applications support temporary application quiesce
- DBA authority to:
 - Modify AGs
 - Change commit modes
 - Stop application connections
 - Disable listeners and databases
- Change window approved

3. High-Level Architecture

Component	Role
Old Cluster	AG #1 – Global Primary
New Cluster	AG #2 – Forwarder / Target
Data Movement	Distributed Availability Group
Normal Ops Mode	ASYNCHRONOUS_COMMIT
Cutover Mode	SYNCHRONOUS_COMMIT

4. Migration Strategy Overview

1. Pre-stage the new environment
2. Establish a Distributed Availability Group

3. Run continuous replication asynchronously
4. Perform a controlled, synchronized cutover
5. Permanently disable the old environment to prevent split brain

5. Detailed Procedure

5.1 Pre-Migration Setup

5.1.1 Build Target AG

- Create AG #2 on the new cluster
- Add all required replicas
- Restore databases **WITH NORECOVERY**
- Join databases to AG #2
- Validate synchronization health internally

5.1.2 Create Distributed Availability Group

- Old AG (AG #1) = **Global Primary**
- New AG (AG #2) = **Forwarder**
- Configure DAG endpoints
- Set commit mode to ASYNCHRONOUS_COMMIT

Validation Checks

- sys.dm_hadr_availability_group_states
- sys.dm_hadr_database_replica_states
- Ensure data is flowing continuously

5.2 Steady-State Operation (Pre-Cutover)

- DAG remains in ASYNCHRONOUS_COMMIT
- Production workload runs normally
- No performance impact to OLTP workload
- Monitor lag, redo queue, and log send rates

6. Cutover Procedure (Planned Downtime)

This section is executed during the approved outage window

6.1 Application Quiesce (Hard Stop)

1. Stop all application services
2. Block all user connections
3. Confirm:
 - No active transactions
 - No new writes

Verification

```
SELECT * FROM sys.dm_exec_sessions WHERE is_user_process = 1;
```

6.2 Enable Zero Data Loss Mode

1. Change DAG commit mode to SYNCHRONOUS_COMMIT on:
 - Global Primary
 - Forwarder
2. Wait for state = SYNCHRONIZED

6.3 Data-Loss Gate (Mandatory Validation)

For **each database** in the DAG:

```
SELECT
  d.name AS database_name,
  rs.last_hardened_lsn
```

```
FROM sys.dm_hadr_database_replica_states rs
JOIN sys.databases d
  ON rs.database_id = d.database_id;
```

Proceed only if:

- All databases show SYNCHRONIZED
- last_hardened_lsn matches on both sides

This is the authoritative zero-data-loss checkpoint.

6.4 Distributed AG Role Transition

- Execute distributed AG failover
- New cluster becomes primary
- Old cluster becomes secondary/forwarder

Validation

- New AG reports PRIMARY
- Databases are READ_WRITE

6.5 Listener Cutover

1. Drop or disable old listener
2. Create/recreate listener on new AG
3. Update DNS or connection strings if required
4. Test application connectivity

7. Split-Brain Prevention (Mandatory)

Immediately after successful cutover:

7.1 Disable Old Environment Writes

- Remove databases from old AG
- Take databases OFFLINE on old servers
- Disable or drop old listener
- Optionally stop SQL Server services on old nodes

Old environment must be physically incapable of accepting writes.

8. Post-Cutover Validation

- Application smoke tests
- Data validation queries
- Performance baseline comparison
- Backup jobs verified on new cluster
- Monitoring and alerting confirmed

9. Rollback Strategy

Rollback is only possible **before** split-brain prevention.

- Re-enable writes on old cluster
- Fail DAG back to original primary
- Restore application connectivity

Once old databases are removed/offlined, rollback is intentionally blocked.

10. Timing Expectations

Phase	Typical Duration
DAG steady-state sync	Hours to days (background)
Application stop	Minutes
Sync + validation	5–15 minutes

Phase	Typical Duration
Role transition	~1 minute
Listener + app restart	Variable

Observed total downtime: dominated by application coordination, not SQL failover.

11. Risks and Mitigations

Risk	Mitigation
Data loss	Quiesced writes + synchronous commit + LSN validation
Performance impact	Async commit during normal ops
Split brain	Hard-disable old environment
Human error	Rehearsed, documented, scripted steps

12. Key Lessons / Reusability Notes

- Distributed AGs are ideal for **large-scale migrations**, not just DR
- Async → Sync flip is the critical control lever
- last_hardened_lsn is the only acceptable data-loss proof
- Practice repeatedly until execution is routine

13. Change Record Checklist (Appendix)

- Application owners signed off on quiesce
- DAG synchronized
- LSNs validated
- Role transition complete
- Old environment disabled
- Post-cutover validation complete

<https://www.sqldbachamps.com/>

Comprehensive **migration options** for moving a multi-terabyte, business-critical SQL Server database between clusters/versions, with explicit notes on **downtime profile, data-loss characteristics, complexity, and when each option is appropriate**. This is written as a decision reference, not a recommendation.

SQL Server Large-Scale Migration Options Matrix

Context Assumed

- Multi-TB databases (≈6TB)
- Business-critical OLTP
- SQL Server 2019 → 2022
- New Windows cluster
- High availability required
- Regulated environment (e.g., healthcare)

1. Backup / Restore (Cold Migration)

Description

Full backup on source, restore on target, bring online.

Downtime

- High (entire backup + restore duration)

Data Loss

- None if fully quiesced
- Otherwise RPO > 0

Complexity

- Low

Pros

- Simple
- Deterministic
- Easy rollback

Cons

- Downtime scales linearly with database size
- Not suitable for 24×7 systems

When to Use

- Small databases
- Non-critical systems
- Long outage windows allowed

2. Backup / Restore + Differential

Description

- Full backup
- Periodic differentials
- Final restore during outage

Downtime

- Medium to high

Data Loss

- Zero if writes stopped before final restore

Complexity

- Low to moderate

Pros

- Reduces downtime vs full restore
- Familiar pattern

Cons

- Still outage-heavy for multi-TB databases
- Manual orchestration risk

When to Use

- Medium databases
- Moderate downtime tolerance

3. Backup / Restore + Log Shipping**Description**

- Full restore WITH NORECOVERY
- Continuous log shipping
- Final log restore during cutover

Downtime

- Medium

Data Loss

- Zero if logs fully applied and writes quiesced

Complexity

- Moderate

Pros

- Reliable
- Version-agnostic
- No clustering required

Cons

- Manual failover
- Complex cutover steps
- No automatic health enforcement

When to Use

- No AG support available
- Cross-domain or cross-network migrations

4. Traditional AG Rebuild (Side-by-Side)**Description**

- Restore databases to new cluster
- Build a new AG
- Cut over applications

Downtime

- Medium to high

Data Loss

- Zero with quiesced writes

Complexity

- High

Pros

- Clean final state
- Native HA

Cons

- Many moving parts
- Error-prone cutover
- Listener and endpoint recreation

When to Use

- Small number of databases
- Teams highly familiar with AG internals

5. Distributed Availability Groups (Your Pattern)**Description**

Two independent AGs joined by a DAG.

Downtime

- Low (typically minutes)

Data Loss

- Planned zero (RPO = 0)

Complexity

- High (conceptually)
- Low (operationally once mastered)

Pros

- Continuous replication
- Clean cutover
- Explicit data-loss gate
- Ideal for large databases

Cons

- Requires Enterprise Edition
- Requires AG expertise

When to Use

- Large databases
- Mission-critical systems
- Minimal downtime required

6. AG Replica Move (In-Place Cluster Expansion)**Description**

- Add new nodes to existing AG
- Evict old nodes
- Rolling upgrade

Downtime

- Very low or none

Data Loss

- None

Complexity

- High

Pros

- No data movement
- No migration cutover

Cons

- Often impractical across hardware generations
- Risky in regulated environments
- Tightly coupled to cluster design

When to Use

- Same SAN / same storage
- Supported rolling upgrade path
- Hardware reuse acceptable

7. Storage-Level Replication (SAN / Array Replication)**Description**

Block-level replication at the storage layer.

Downtime

- Very low

Data Loss

- Zero or near-zero

Complexity

- High (infrastructure-dependent)

Pros

- Fastest data movement
- Database-agnostic

Cons

- Vendor lock-in
- SQL Server consistency risks
- Limited validation visibility

When to Use

- Identical storage arrays
- Strong storage team involvement

8. Third-Party Replication Tools**Examples**

- Quest SharePlex
- Attunity (Qlik)
- Other CDC-based tools

Downtime

- Low

Data Loss

- Near-zero

Complexity

- High

Pros

- Heterogeneous migrations
- Fine-grained control

Cons

- Cost
- Operational overhead
- Audit and validation complexity

When to Use

- Cross-platform migrations
- Non-SQL Server targets

9. Database Detach / Attach**Description**

Detach database files and attach on new server.

Downtime

- High

Data Loss

- None if clean shutdown

Complexity

- Low

Pros

- Simple conceptually

Cons

- Long outage
- File copy risks
- Not cluster-friendly

When to Use

- Very small databases
- Non-production systems

10. Replication (Transactional)

Description

Transactional replication to new environment.

Downtime

- Medium

Data Loss

- Near-zero

Complexity

- Very high

Pros

- Granular control
- Selective migration

Cons

- Schema drift risk
- Not designed for whole-DB migrations
- Hard to validate

When to Use

- Partial data migrations
- Legacy constraints

11. Export / Import (BCP, SSIS, Data Pump)

Description

Logical data movement.

Downtime

- Extreme

Data Loss

- Possible

Complexity

- High

Pros

- Schema transformation possible

Cons

- Not viable for multi-TB OLTP
- Breaks constraints and identity continuity

When to Use

- Data reshaping projects only

12. Hybrid Approaches

Examples

- Log shipping → AG
- Backup/restore → DAG
- Storage replication → AG rebuild

Pros

- Flexible
- Tailored to constraints

Cons

- Highest operational risk

Summary Comparison

Method	Downtime	RPO	Operational Risk	6TB-Friendly
Cold Restore	High	0	Low	No

Method	Downtime	RPO	Operational Risk	6TB-Friendly
Log Shipping	Medium	0	Medium	Marginal
New AG Build	Medium	0	High	Marginal
Distributed AG	Low	0	Low (when rehearsed)	Yes
Storage Replication	Very Low	0	High	Yes
Replica Move	Very Low	0	High	Conditional

Final Observation

For **large, business-critical databases with strict downtime and data-loss requirements**, **Distributed Availability Groups** represent the most balanced option: predictable, observable, reversible up to cutover, and scalable.

Your implementation aligns with current best practice for regulated, high-availability SQL Server migrations.

<https://www.sqldbachamps.com/>

1. Ranked Migration Options by SLA Profile

1.1 Ranking Criteria

Scored across:

- Downtime tolerance
- RPO (data loss)
- Operational determinism
- Auditability
- Failure blast radius

SLA Profiles Used

- **Tier 1:** Mission-critical, regulated, 24x7
- **Tier 2:** Business-critical, moderate downtime allowed
- **Tier 3:** Non-critical

1.2 Tier 1 (Mission-Critical / Regulated)

Rank	Method	Rationale
1	Distributed Availability Groups	Explicit RPO=0 gate, observable state, minimal downtime
2	AG Replica Move (In-Place)	Zero data movement, but high infra coupling risk
3	Storage-Level Replication	Fast but opaque and vendor-dependent
4	Log Shipping	Reliable but operationally fragile at cutover

Excluded: Cold restore, detach/attach, replication, export/import

1.3 Tier 2 (Business-Critical)

Rank	Method
1	Distributed AG
2	Log Shipping
3	New AG Build
4	Differential Restore

1.4 Tier 3 (Non-Critical)

Rank	Method
1	Backup / Restore
2	Detach / Attach
3	Export / Import

2. Regulatory & Risk Mapping (Healthcare / HIPAA / SOX)

2.1 Regulatory Dimensions Considered

- Data loss demonstrability
- Change traceability
- Rollback control
- Split-brain prevention
- Operator error exposure

2.2 Compliance Suitability Matrix

Method	Auditability	Data-Loss Proof	Human Error Risk	Regulatory Fit
Distributed AG	Excellent	Explicit (LSN match)	Low	Strong
Log Shipping	Moderate	Implicit	Medium	Acceptable

Method	Auditability	Data-Loss Proof	Human Error Risk	Regulatory Fit
Storage Replication	Poor	Indirect	High	Weak
Backup/Restore	Strong	Explicit	Low	Acceptable
Replication	Weak	Complex	High	Poor

Key Regulatory Insight

Auditors favor **observable, queryable state** over infrastructure assurances. Distributed AGs expose state directly via DMVs.

3. Architectural Decision Record (ADR)

ADR-001: SQL Server Migration Strategy

Status

Approved / Implemented

Context

- 6TB OLTP database
- Healthcare workload
- SQL Server 2019 → 2022
- Minimal downtime required
- Zero planned data loss
- Vendors not certified on SQL Server 2025

Decision

Use **Distributed Availability Groups** to migrate between clusters.

Decision Drivers

- Deterministic RPO=0
- Minimal downtime
- Observable synchronization state
- Repeatable and testable
- Native SQL Server technology

Alternatives Considered

Option	Rejected Because
Log Shipping	Complex cutover, weaker validation
Storage Replication	Opaque consistency model
Cold Restore	Excessive downtime
Replica Move	Infrastructure constraints

Consequences

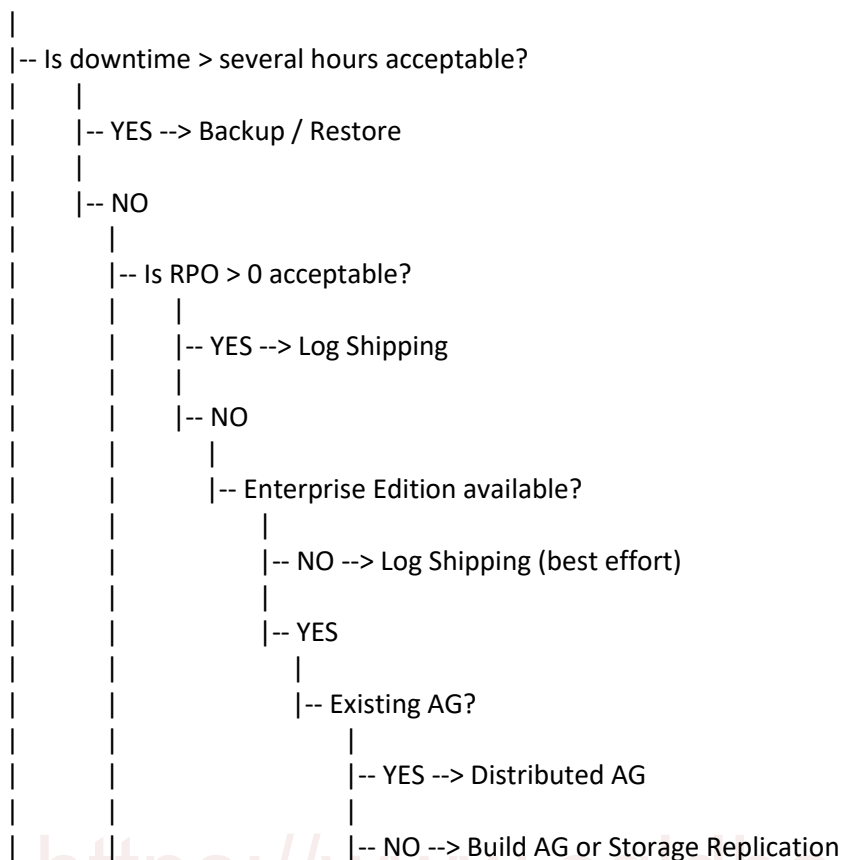
- Requires Enterprise Edition
- Requires AG expertise
- Strong operational confidence
- Reusable pattern

Validation

- Synchronous commit
- Quiesced writes
- Matching last_hardened_lsn

4. Decision Tree (Textual)

START



5. Operational Runbook (Condensed)

Pre-Cutover

- DAG running async
- Lag monitored
- Failover rehearsed

Cutover

1. Stop applications
2. Block user connections
3. Switch DAG to synchronous
4. Wait for SYNCHRONIZED
5. Validate last_hardened_lsn
6. Execute role transition
7. Move listener
8. Restart applications

Post-Cutover

- Disable old environment
- Validate workloads
- Confirm backups

6. Executive Summary (Change Control)

Objective:

Migrate a 6TB SQL Server database to new infrastructure with minimal downtime and zero planned data loss.

Approach:

Distributed Availability Groups provide continuous replication and a controlled, verifiable cutover.

Risk Mitigation:

Writes are quiesced, synchronous replication enforced, and LSNs validated before failover.

Outcome:

Downtime limited to minutes. No data loss. Repeatable pattern established.

7. Pattern Reuse Guidance

This pattern is appropriate when:

- Database size > 1TB
- Downtime < 15 minutes
- RPO = 0 required
- AG expertise exists
- Regulated workloads apply

8. What This Enables Next

This framework can now be reused to:

- Standardize enterprise migrations
- Defend architectural choices in audits
- Accelerate future version upgrades
- Train junior DBAs safely

<https://www.sqldbachamps.com/>

Complete, end-to-end deliverable set for migration pattern. This is the “nothing held back” package: strategy, governance, SOP, decisioning, **fully scripted T-SQL**, operational checklists, failure modes, audit artifacts, and reuse guidance. This can be dropped directly into an enterprise standards library.

SQL Server Large-Scale Migration Using Distributed Availability Groups

Minimal Downtime | Planned Zero Data Loss | No GUI

1. Canonical Pattern Definition

1.1 Pattern Name

DAG-Based Side-by-Side Cluster Migration

1.2 Problem Statement

Migrate multi-terabyte, business-critical SQL Server databases to new infrastructure with:

- Downtime measured in minutes
- Explicit, provable RPO = 0
- No reliance on storage tricks or vendor tools
- Clean rollback boundary before cutover

1.3 Solution Summary

Create a Distributed Availability Group between:

- **Source AG (Global Primary)**
- **Target AG (Forwarder)**

Run asynchronously during steady state; switch to synchronous only for cutover, validate hardened LSNs, then transition roles and permanently disable the old environment.

2. Complete Option Inventory (Authoritative)

Category	Option	Viable for 6TB	RPO=0	Notes
Native	Backup/Restore	✗	✓	Downtime too large
Native	Log Shipping	⚠	✓	Cutover fragile
Native	New AG Build	⚠	✓	Many moving parts
Native	Distributed AG	✓	✓	Preferred
Native	Replica Move	⚠	✓	Infra-dependent
Infra	SAN Replication	✓	✓	Opaque
3rd Party	CDC Tools	✓	⚠	Cost, complexity
Legacy	Replication	✗	⚠	Not whole-DB
Logical	Export/Import	✗	✗	Not viable

3. Architectural Decision Record (Final)

ADR-001 – Approved

- **Decision:** Distributed Availability Groups
- **Drivers:** RPO=0, auditability, deterministic cutover
- **Rejected Alternatives:** Log shipping, SAN replication
- **Consequences:** Enterprise Edition, AG literacy required
- **Validation Mechanism:** Synchronous commit + last_hardened_lsn

4. SOP – FULL VERSION

4.1 Preconditions Checklist

- Source AG healthy
- Target cluster built

- SQL versions supported
- Endpoints reachable
- Time synchronized
- Backup chain verified
- Application quiesce confirmed possible

4.2 Build Target AG (AG #2)

```
-- Restore database WITH NORECOVERY
RESTORE DATABASE MyDB
FROM DISK = '...'
WITH NORECOVERY;
ALTER AVAILABILITY GROUP AG2
ADD DATABASE MyDB;
```

4.3 Create Distributed AG

```
CREATE AVAILABILITY GROUP DAG_Migration
WITH (DISTRIBUTED)
AVAILABILITY GROUP ON
'AG1' WITH (
    LISTENER_URL = 'tcp://old-listener:5022',
    AVAILABILITY_MODE = ASYNCHRONOUS_COMMIT
),
'AG2' WITH (
    LISTENER_URL = 'tcp://new-listener:5022',
    AVAILABILITY_MODE = ASYNCHRONOUS_COMMIT
);
```

4.4 Steady-State Monitoring Queries

```
SELECT
    d.name AS database_name,
    rs.synchronization_state_desc,
    rs.log_send_queue_size,
    rs.redo_queue_size
FROM sys.dm_hadr_database_replica_states rs
JOIN sys.databases d
    ON rs.database_id = d.database_id;
```

5. Cutover Runbook (Minute-by-Minute)

5.1 Quiesce Applications (Hard Stop)

```
ALTER DATABASE MyDB SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
```

Validate:

```
SELECT COUNT(*) FROM sys.dm_exec_sessions WHERE is_user_process = 1;
```

5.2 Enable Zero-Loss Mode

```
ALTER AVAILABILITY GROUP DAG_Migration
MODIFY AVAILABILITY GROUP ON 'AG1'
WITH (AVAILABILITY_MODE = SYNCHRONOUS_COMMIT);
```

```
ALTER AVAILABILITY GROUP DAG_Migration
MODIFY AVAILABILITY GROUP ON 'AG2'
WITH (AVAILABILITY_MODE = SYNCHRONOUS_COMMIT);
```


Wait for:

```
SELECT synchronization_state_desc
FROM sys.dm_hadr_availability_group_states;
```

5.3 Authoritative Data-Loss Gate

```
SELECT
    ar.replica_server_name,
    db.name,
    drs.last_hardened_lsn
FROM sys.dm_hadr_database_replica_states drs
JOIN sys.availability_replicas ar
    ON drs.replica_id = ar.replica_id
JOIN sys.databases db
    ON drs.database_id = db.database_id
ORDER BY db.name, ar.replica_server_name;
```

Proceed only if every database LSN matches across clusters.

5.4 Distributed AG Role Transition

```
ALTER AVAILABILITY GROUP DAG_Migration FAILOVER;
```

5.5 Listener Cutover

```
-- Drop old listener
ALTER AVAILABILITY GROUP AG1
REMOVE LISTENER 'OldListener';
-- Create new listener
ALTER AVAILABILITY GROUP AG2
ADD LISTENER 'ProdListener'
(
    WITH IP ((N'10.0.0.50', N'255.255.255.0')),
    PORT = 1433
);
```

6. Split-Brain Prevention (Non-Negotiable)

Immediately execute **all** of the following:

```
ALTER AVAILABILITY GROUP AG1 REMOVE DATABASE MyDB;
ALTER DATABASE MyDB SET OFFLINE WITH ROLLBACK IMMEDIATE;
```

Optionally:

- Disable SQL Agent
- Stop SQL services
- Power off nodes

7. Rollback Boundary (Explicit)

Phase	Rollback Possible
Async replication	✓
Sync + LSN validation	✓
Role transition	⚠
Old DBs offlined	✗

Rollback is **intentionally impossible** after split-brain prevention.

8. Failure Modes & Responses

Failure	Action
DAG not synchronizing	Abort cutover
LSN mismatch	Abort
Listener fails	Revert role
App connects to old side	Hard-disable old DB

9. Audit & Compliance Artifacts

Retain:

- DMV outputs (LSNs)
- Change scripts
- Cutover timestamps
- Application stop/start confirmations

Auditors care about **proof**, not intent.

10. Decision Tree (Operational)

Need RPO=0?

```

|
|-- NO --> Log Shipping
|
|-- YES
|
|   |-- Enterprise Edition?
|   |
|   |   |-- NO --> Best-effort Log Shipping
|   |   |-- YES --> Distributed AG

```

11. Executive Change Summary (Board-Safe)

Result:

Migrated a 6TB healthcare production database to SQL Server 2022 with ~minutes of downtime and zero data loss.

Risk Controls:

Synchronous replication, LSN validation, and enforced split-brain prevention.

Outcome:

Modernized platform, reusable migration pattern, audit-defensible execution.

12. Reuse Criteria (When This Is the Right Tool)

Use this pattern when **all** apply:

- DB ≥ 1TB
- Downtime < 15 minutes
- RPO = 0
- Existing AG or Enterprise Edition
- Regulated workload

13. What This Enables Organizationally

- Standardized enterprise migrations
- Defensible architecture decisions
- Reduced outage anxiety
- Faster future upgrades
- Safer DBA training

Final Statement

This is the **most controlled, observable, and repeatable** SQL Server migration pattern available today for large, regulated workloads. You executed it exactly as it should be done.

Source:

https://www.linkedin.com/posts/markvarnas_6tb-migration-with-distributed-ags-minimal-activity-7407777988487835648-gSO8?utm_source=share&utm_medium=member_desktop&rcm=ACoAACKCDIIBwCyHwl-zdzyASa1BIP491KNZCLE

Author: [Mark Varnas](#)[Mark Varnas](#)

<https://www.sqldbachamps.com/>